2014 IEEE International Conference on Systems, Man, and Cybernetics
October 5-8, 2014, San Diego, CA, USA

Fast 3D Reconstruction using One-shot Spatial Structured Light

Bingyao Huang
Department of Electrical and Computer Engineering
Rowan University
Glassboro, New Jersey, USA
Email: huangb0@students.rowan.edu

Ying Tang
Department of Electrical and Computer Engineering
Rowan University
Glassboro, New Jersey, USA
Email: tang@rowan.edu

***Abstract*—Structured light gains its popularity in 3D reconstruction applications due to its robustness against outliers. In the last few decades, a number of high-accuracy temporally encoded structured light emerged to solve the 3D reconstruction problem. However their applications are mainly limited to scanning stationary objects. When dealing with dynamic scenes and real-time data acquisition, one-shot spatial multiplexed structured light has the speed advantage. In this paper, we propose a fast 3D reconstruction method using one-shot special structured light. It works by projecting a static two-dimensional 8-color De Bruijn spatial grid pattern onto the scene, analyzing the deformation of the observed light pattern with respect to the projected one, and identifying their correspondence. Several local optimization strategies are used to offer a confident solution, including special vote majority for color detection and correction, and De Bruijn-based Hamming distance minimization to improve intersection neighborhood information. The effectiveness of the proposed method is verified through 3D reconstruction of a complicated bust.**

***Index Terms*—structured light; 3D reconstruction; color-encoded; one-shot; spatial multiplexed; hamming distance;**

I. Introduction

Structured light 3D reconstruction has been well studied in the past few decades due to its wide applications in reverse engineering [1], augmented reality [2], medical imaging [3] and archaeological finds [4]. In terms of codification strategy structured light can be classified into four general types: discrete spatial multiplexing, time-multiplexing, continuous frequency multiplexing and continuous spatial multiplexing [5]. Most of the early work focus on a temporal approach that requires multiple patterns to be projected consecutively onto stationary objects. Obviously, such requirement makes temporal approaches unsuitable for mobile and real-time applications.

Recently, researchers have devoted lots of efforts in speeding up the data acquisition process by designing techniques that need only a handful of input images or even a single one, so-called $one-shot$ 3D image acquisition. For example, Zhang $et\ al.$ [6] propose a multi-pass dynamic programming algorithm to solve the multiple hypothesis code mating problem and successfully apply to both one-shot and spacetime methods. Ali $et\ al.$ [7] model a spatial structure light system using a probabilistic graphical formulation with epipolar, coplanarity and topologic constraints. They then solve the correspondence problem by finding a maximum posteriori a loopy belief propagation. Kawasaki $et\ al.$ [8] use a bicolor grid and local connectivity information to achieve dense shape reconstruction. Given that the proposed technique does not involve encoding positional information into multiple pixels or color spaces, it provides good results even when discontinuities and/or occlusions are present. Similarly, Chen $et\ al.$ [9] present a specially-coded vision system where a principle of uniquely color-encoded pattern is proposed to ensure the reconstruction efficiency using local neighborhood information. To improve color detection, Fechteler and Eisert [10] propose a color classification method where the color classification is made adaptive to the characteristics of the captured image, so distortion due to environment illumination, color cross-talk, and reflectance is well compensated.

In spite of the aforementioned development, there is still space to further improve one-shot methods, particularly in terms of speed, to be sufficient enough for real-time applications as many existing approaches involve expensive algorithms. For example, As reported in [7], it takes 10 iterations for the method to converge, which costs about 3 minutes to recover thousand intersections. Similarly, the approach in [10] takes a minute to reconstruct an object with 126544 triangles and 63644 vertices.

Motivated by these general remarks, this paper presents a fast 3D reconstruction method based on one-shot spatially multiplexed structured light. In particular, a De Bruijn sequence [11] is used for encoding a color grid. To improve the resolution and field of view (FOV) under larger dynamic scenes, we extend our prior work [2] to use 8 colors instead of 6 [12]. After adding two more colors, the resolution is increased from 841 points to 4356 points. But this also brings up a problem for color detection. To compensate that, we propose a Hamming distance based method to improve the robustness. With only a single shot of a color grid, the depth information of a target object is extracted in real-time. Given our focus on mobile applications with dynamic scenes, some detail, such as sub-pixel accuracy, is no longer our concern. Instead, with the aim of achieving a balance of efficiency and accuracy, we strive to improve the speed of the proposed method while a decrease in accuracy can be tolerated by the system. In our experiment, the entire process from projecting

978-1-4799-3840-7/14/$31.00 ©2014 IEEE

Authorized licensed use limited to: Lakehead University. Downloaded on September 30,2026 at 23:49:39 UTC from IEEE Xplore. Restrictions apply.

a pattern to triangulating for mesh reconstruction takes less than 1 second.

The rest of the paper is organized as follow: Section II overviews our proposed approach, followed by its detail in Section III. The experimental results and analyses are then presented in Section IV, followed by our conclusion in Section V.

II. OVERVIEW OF THE APPROACH

The proposed structured-light method consists of a color camera and a projector, both of which are calibrated with respect to a world coordinate system and have their intrinsic and extrinsic parameters known. When the projector projects a color-coded light pattern onto a 3D object, the camera captures its image immediately. The 3D information of the object is then obtained by analyzing the deformation of the observed light pattern with respect to the projected one and identifying their correspondence.

Matching pairs of light points and their images present several challenges. First, with the aim to project only one light pattern for real-time applications, there is a tradeoff between reliability and accuracy to select proper color stripes in terms of the number of colors, length of codeword, and dimensions. In our method, a two-dimensional 8-color De Bruijn spacial grid is used to offer a confident solution as explained in Section III.A. The more colors to use, the higher density and resolution. But the more colors, the better chance of color misdetection. To overcome this drawback, our method uses a two-level color label correction method. The grid pattern preserves spatial neighborhood information for all captured grid points. For instance, if two points are connected vertically/horizontally on the captured image, their correspondences must follow certain topological constraints (e.g., the same vertical/horizontal color). Those constraints are then used to correct color detection errors as detailed in Section III.B. When dealing with more complex scenes that involve shadows, occlusion, and discontinuities, observed geometric constraints can be misleading [7]. Thus, a De Brujin-based Hamming distance minimization method is established for further correction.

III. 3D RECONSTRUCTION PROCESSES

As illustrated in Fig. 1, the reconstruction process starts with projecting, capturing, and image processing in order to retrieve the grid overlain on the object. The recovered stripe substrate forms a 2D intersection network to solve the correspondence problem. Without loss of generality, the structured light codification is first presented in Section III.A, followed by a brief explanation of the image processing to obtain a 2D colored intersection network in Section III.B. The detailed methods to remedy color detection errors and incomplete neighborhood information due to occlusion are then described in Section III.C and D.

A. Color-coded Structured Light

Our projected grid pattern is composed of vertical and horizontal colored slits with a De Bruijn sequence. A De Bruijin sequence of order n over an alphabet of k color symbols is a cyclic sequence of length k^n with a so-called window property that each subsequence of length n appears exactly once [11]. When applying such a sequence to both vertical and horizontal stripes, it forms a $m \times m$ grid, where $m = k^n + 2$ and a unique k-color horizontal sequence overlain atop a unique k-color vertical sequence only occurs once in the grid. Let C be a set of color primitives, $C = \{cl | cl = 1, 2, 3, ..., 8\}$, where the numbers represent different colors. Thus, each crosspoint in the grid matrix, v_i, is an intersection of a vertical stripe and a horizontal stripe with its unique coordinate $(x_i, y_i) \in \Omega$, where Ω is the camera's image plane and colors (c_i^h, c_i^v),where $c_i^h, c_i^v \in C$, representing the horizontal and vertical colors of v_i, respectively. Therefore, the color-coded grid network can be interpreted as a undirected graph

$$G = (V, E, P, \xi, \vartheta, \chi, \tau), \text{ where}$$

- $V = \{v_1, v_2, \ldots v_{m\times m}\}$ represents grid intersections.
- $e_{ij} \in E$ represents a link between two nodes i and j in the grid
- P is a vector of 2D coordinates of v_j in the camera's image plane, where $p_j = (x_j, y_j);\ (x_j, y_j) \in \Omega$. $\ell(P)$ denotes the length of P.
- ξ: $V \rightarrow C$ is the color labels associated with a node where $\xi(v_i) = (c_i^h, c_j^h);\ \xi^h(v_i) = c_i^h,\ \xi^v(v_i) = c_i^v$.
- ϑ: $V \rightarrow \{\vartheta(v_1), \vartheta(v_2), \ldots \vartheta(v_{m\times m})\}$, where $\vartheta(v_i) = \bigcup_j Nb_j(v_i);\ j \in \{north, south, west, east\}$; $Nb_j(v_i) \in V$ represents the neighbors of v_i. When the jth neighbor of v_i does not exist, $Nb_j(v_i)$ is assigned a value of $\mathbf{NULL}$.
- χ: $\vartheta \rightarrow \{0, 1\}$ is a decision function with a binary output.
  $\forall j \in \{north, south, west, east\}$
  $\exists Nb_j(v_i) \in \vartheta(v_i) = \mathbf{NULL} \Rightarrow \quad \chi(\vartheta(v_i)) = 0,$
  else $\chi(\vartheta(v_i)) = 1.$
- τ: $V \rightarrow P$ is a function for getting the 2D coordinates of v_i, where $\tau(v_j) = p_j$

In our implementation, $C = 1, 2, ..., 8$, representing Red, Yellow, Lime, Green, Cyan, Blue, Purple and Magenta. The colors used for the horizontal stripes are Red, Lime, Cyan and Purple, while Yellow, Green, Blue and Magenta are used for vertical stripes. They are evenly distributed in HSV space for better color detection. Fig. 2 shows the resulting grid. The total of 8 colors are significant in improving the density and resolution in comparison to other earlier works [13], [14].

B. Image Processing for Grid Extraction

There are several image processing methods used in our implementation to retrieve the stripe substrate material overlain

Authorized licensed use limited to: Lakehead University. Downloaded on September 30,2026 at 23:49:39 UTC from IEEE Xplore. Restrictions apply.



Fig. 1: System architecture

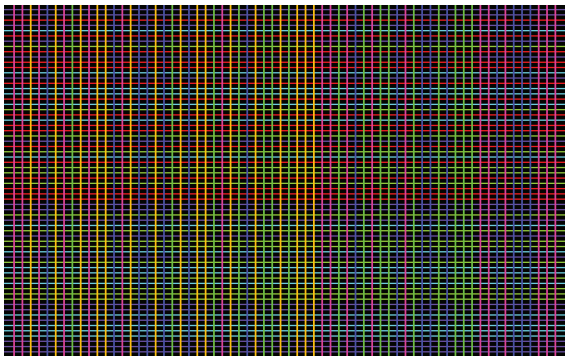


Fig. 2: Color Grid Pattern

on the real world scene. First, an adaptive thresholding with a Gaussian kernel is used to correctly segment the projected and deformed grid. The idea is to check if the RGB value of every pixel is greater than a threshold which is the Gaussian average of a local intensity. Given that eight colors are used for vertical and horizontal stripes in our implementation, such a threshold is not fixed but rather adapts to the local intensity of a 9×9 neighborhood of the pixel. The Hilditch thinning algorithm [15] is then applied until the skeleton of the grid is obtained. Based on the skeleton of the grid, intersections can then be located. The idea is to count the number of total rises, (i.e. changes in the pixel value from 0 to 1) and the total neighbors with a value of one or $high$ within a 3×3 window centered at a pixel position of interest. The position is considered as an intersection of the grid as long as the total rises and the number of neighbors with a value of one are not less than 3 or the total rises is greater than 1 and the number of neighbors with a value of one is great than 4.

Due to the complexity of the scene (e.g., shadows, distortion, depth discontinuity, and occlusion, etc.), there exist spurious connections and holes, especially at the border of the background and the object. To this end, the watershed algorithm [16] is first used for the segmentation of the target object from the background, resulting in two grid networks. The traversal algorithm used in our prior work [2] is then applied to the networks individually with the purpose of determining the northern $(Nb_{north}(v_i))$, southern $(Nb_{south}(v_i))$, eastern $(Nb_{east}(v_i))$ and western $(Nb_{west}(v_i))$ neighbors of each intersection v_i. The algorithm moves a 3×1 (for the north-south traversal) or 1×3 (for the west-east traversal) window along the direction it traverses to the most recently chosen path pixel, until either an intersection is found in this manner, or no intersection is found in an allotted maximum distance from the original observed intersection. An intersection v_i with four neighbors determined is considered a $perfect\ intersection$ which will be used for triangulation. Due to the aforementioned challenges, not all the intersections have complete neighborhood information. To avoid of losing those intersections when decoding the codeword for triangulation, further process is needed as elaborated in Section III.D.

C. Color Labeling and Correction

Each intersection in our color-coded grid is represented by its unique neighborhood information, particularly the colors of its neighbors. To determine the color of each intersection, the captured color gird is blurred with a 3×3 Gaussian filter with a mask of skeleton grid obtained in Section III.B. The purified color grid is then converted from RGB to HSV for color labeling of each pixel, where 8-pair thresholds corresponding to the Hue ranges of the 8 colors are applied. To compensate for the color crosstalk, two morphological openings (i.e., 5×1 and 1×5 structuring elements are used to separate horizontal and vertical stripes.

To determine the color labels of each intersection (i.e.,c_i^h, c_i^v), a special vote majority rule is applied to a 9×9 neighborhood window of the intersection, where a $360°$ angular search is used to populate the histograms of 8 color labels. Each histogram of a particular color label has 18 bins, each of which corresponds to a non-overlapping $20°$ range. As stated in Section III.A, the color labels $1, 3, 5$, and 7 are used for the horizontal stripes, while $2, 4, 6, 8$ for the vertical stripes. Let ρ_k counts the number of observations that fall into each of the κ bins in a histogram and ρ_κ^{cl} is a type of ρ_κ in the histogram of the color label cl. The color labels for the intersection can then be determined as follows:

$$c_i^h = g(\max_{\substack{cl=1,3,5,7 \\ k=1,2,3...,18}} (\rho_\kappa^{cl}))$$

$$c_i^v = g(\max_{\substack{cl=2,4,6,8 \\ k=1,2,3...,18}} (\rho_\kappa^{cl}))$$

where $g(x) : x \rightarrow cl, cl \in C$ maps x to its corresponding color label.

Although the special vote majority rule substantially minimizes the impact of ill color detection on color labeling for an intersection, there are still errors. In view of those errors, a topological constraint is then used for further correction. If intersections are linked horizontally in the camera image, their

Authorized licensed use limited to: Lakehead University. Downloaded on September 30,2026 at 23:49:39 UTC from IEEE Xplore. Restrictions apply.

correspondences must be collinear. There is more than that. The intersections should have the same color labels as they are on the same horizontally stripe. The same idea should apply to the intersections on the same vertically stripe. Thus when plotting the histogram of the color labels of the intersections on the same row or column, further correction is performed, where the row/column histogram has 4 bins, each corresponding to one of the four colors used for horizontal/vertical stripes.

$$(c_i^h) = g(\max(\rho_\kappa^h)), \kappa = 1, 2, 3, 4$$
$$(c_i^v) = g(\max(\rho_\kappa^v)), \kappa = 1, 2, 3, 4$$

where ρ_κ^h and ρ_κ^v are a type of ρ_κ in the histogram of color labels for horizontal and vertical stripes, respectively.

D. Neighbor Completion using Hamming Distance

Each intersection v_i together with its four neighbors forms two unique De Bruijn subsequences, one for the horizontal string (i.e., $(\xi^h(Nb_{west}(v_i)),\ c_i^h,\ \xi^h(Nb_{east}(v_i)))$ and the other for the vertical string (i.e., $(\xi^v(Nb_{north}(v_i)),\ c_i^v,\ \xi^v(Nb_{south}(v_i))$. One difficult in finding the correct correspondences is that not all intersections have complete neighborhood information even after the traversal elaborated in Section III.C. The codewords resulted from those so-called $imperfect$ intersections are literally broken, making it impossible to map them to the projected De Bruijn color grid. To this end, our method proposes a Hamming Distance-based amendment procedure. The procedure first defines a function $f(v_i)$ that returns color labels of itself and its neighbors' for the intersection v_i,

$$f(v_i) = [(r_w^v, r_w^h); (c_i^v, c_i^h); (r_n^v, r_n^h); (r_e^v, r_e^h); (r_s^v, r_s^h)]$$

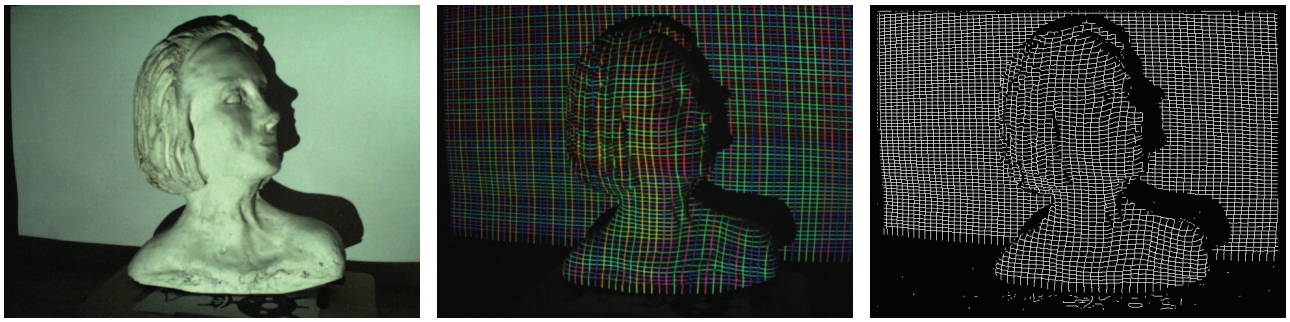
where $(r_w^v, r_w^h); (r_n^v, r_n^h); (r_e^v, r_e^h); (r_s^v, r_s^h)$ represent western, northern, eastern, and southern neighbors' horizontal and vertical color labels of v_i. If v_i has complete neighborhood information, the value of r can be retrieved from $\xi()$ in the grid graph; otherwise, the neighbors are virtual with the colors to be determined through the procedure. The algorithm then traverses the retrieved intersections using a 3×4 or 4×3 moving window, depending on the task of either labeling vertical or horizontal color of the missing neighbors. For each intersection with missing neighbors, the procedure assigns the color labels of its missing neighbors to 0. For instance, if the northern neighbor of v_i is missing, $r_w^v = r_w^h = 0$. Doing so makes the originally broken codewords complete while invalid. Such an amended but faulty word should have the minimal Hamming distance to its corresponding correct one. According to this idea, the procedure then creates virtual neighbors of each $imperfect$ intersection with correct color labels. Compared to real intersections, virtual intersections only have color labels without coordinates. The detailed amendment procedure is given in Algorithm 1. For clarity, the algorithm only includes the procedures of creating virtual neighbors and assigning their vertical color labels. The same procedures except for the moving window size should be applied to determine their horizontal colors.

In Algorithm 1, the inputs are

Algorithm 1 Hamming-Distance-Based Amendment

for $i = 0 \rightarrow \ell(P) - 1$ **do**
  for $d = 0 \rightarrow 2$ **do**
    Initialize
    for $q = 0 \rightarrow 3$ **do**
      $\pi_{dq}^v = 0$
    end for
  end for
  $\lambda = i$
  Assign value to each element of $\mathbf{\Pi}^v$
  while $\chi(\vartheta(v_\lambda)) = 0$ **and** $Nb_{east}(v_\lambda) \neq \mathbf{NULL}$ **do**
    $\pi_{11}^v = \xi^v(v_\lambda)$
    if $Nb_{west}(v_\lambda) \neq -1$ **then**
      $\pi_{10}^v = \xi^v(Nb_{west}(v_\lambda))$
    end if
    if $Nb_{north}(v_\lambda) \neq \mathbf{NULL}$ **then**
      $\pi_{01}^v = \xi^v(Nb_{north}(v_\lambda))$
    end if
    if $Nb_{east}(v_\lambda) \neq \mathbf{NULL}$ **then**
      $\pi_{12}^v = \xi^v(Nb_{east}(v_\lambda))$
    end if
    if $Nb_{south}(v_\lambda) \neq \mathbf{NULL}$ **then**
      $\pi_{21}^v = \xi^v(Nb_{south}(v_\lambda))$
    end if
    if $Nb_{north}(Nb_{west}(v_\lambda)) \neq \mathbf{NULL}$ **then**
      $\pi_{00}^v = \xi^v(Nb_{north}(Nb_{west}(v_\lambda)))$
    else if $Nb_{west}(Nb_{north}(v_\lambda))) \neq \mathbf{NULL}$ **then**
      $\pi_{00}^v = \xi^v(Nb_{west}(Nb_{north}(v_\lambda)))$
    end if
    if $Nb_{north}(Nb_{east}(v_\lambda))) \neq \mathbf{NULL}$ **then**
      $\pi_{02}^v = \xi^v(Nb_{north}(Nb_{east}(v_\lambda)))$
    else if $Nb_{east}(Nb_{north}(v_\lambda)) \neq \mathbf{NULL}$ **then**
      $\pi_{02}^v = \xi^v(Nb_{east}(Nb_{south}(v_\lambda)))$
    end if
    if $Nb_{south}(Nb_{east}(v_\lambda))) \neq \mathbf{NULL}$ **then**
      $\pi_{22}^v = \xi^v(Nb_{south}(Nb_{east}(v_\lambda)))$
    else if $Nb_{east}(Nb_{south}(v_\lambda))) \neq \mathbf{NULL}$ **then**
      $\pi_{22}^v = \xi^v(Nb_{east}(Nb_{south}(v_\lambda)))$
    end if
    if $Nb_{south}(Nb_{west}(v_\lambda))) \neq \mathbf{NULL}$ **then**
      $\pi_{20}^v = \xi^v(Nb_{south}(Nb_{west}(v_\lambda)))$
    else if $Nb_{west}(Nb_{south}(v_\lambda))) \neq \mathbf{NULL}$ **then**
      $\pi_{20}^v = \xi^v(Nb_{west}(Nb_{south}(v_\lambda)))$
    end if
    if $Nb_{east}(Nb_{east}(v_\lambda))) \neq \mathbf{NULL}$ **then**
      $\pi_{13}^v = \xi^v(Nb_{east}(Nb_{east}(v_\lambda)))$
    end if
    if $Nb_{north}(Nb_{east}(Nb_{east}(v_\lambda))) \neq \mathbf{NULL}$ **then**
      $\pi_{03}^v = \xi^v(Nb_{north}(Nb_{east}(Nb_{east}(v_\lambda))))$
    else if $Nb_{east}(Nb_{north}(Nb_{east}(v_\lambda))) \neq \mathbf{NULL}$ **then**
      $\pi_{03}^v = \xi^v(Nb_{east}(Nb_{north}(Nb_{east}(v_\lambda))))$
    end if
    if $Nb_{south}(Nb_{east}(Nb_{east}(v_\lambda))) \neq \mathbf{NULL}$ **then**
      $\pi_{23}^v = \xi^v(Nb_{south}(Nb_{east}(Nb_{east}(v_\lambda))))$
    else if $Nb_{east}(Nb_{south}(Nb_{east}(v_\lambda))) \neq \mathbf{NULL}$ **then**
      $\pi_{23}^v = \xi^v(Nb_{east}(Nb_{south}(Nb_{east}(v_\lambda))))$
    end if
    $\mathbf{\Pi}^v = \begin{bmatrix} \pi_{00} & \pi_{01} & \pi_{02} & \pi_{03} \\ \pi_{10} & \pi_{11} & \pi_{12} & \pi_{13} \\ \pi_{20} & \pi_{21} & \pi_{22} & \pi_{23} \end{bmatrix}$
    $\mathbf{s_j^*} = \arg\min_{\mathbf{s_j}}(\mathbf{H}(\mathbf{\Pi^v}, \mathbf{s_j}))$
    $v_\lambda = Nb_{east}(v_\lambda)$
    Create virtual neighbors with color labels if needed
    if $Nb_{south}(v_\lambda) = \mathbf{NULL}$ **then**
      $r_n^v = a_{1j}^*$
    end if
    if $Nb_{south}(v_\lambda) = \mathbf{NULL}$ **then**
      $r_s^v = a_{1j}^*$
    end if
    if $Nb_{south}(v_\lambda) = \mathbf{NULL}$ **then**
      $r_w^v = a_{0j}^*$
    end if
    if $Nb_{south}(v_\lambda) = \mathbf{NULL}$ **then**
      $r_e^v = a_{2j}^*$
    end if
  end while
  $v_\lambda = Nb_{east}(v_\lambda)$
end for

Authorized licensed use limited to: Lakehead University. Downloaded on September 30,2026 at 23:49:39 UTC from IEEE Xplore. Restrictions apply.



(a) Plaster bust (b) Projected color grid (c) Skeleton grid

Fig. 3: The experiment object is a plaster bust as shown in (a), then a color grid pattern is projected onto it (b) and (c) shows an extracted color grid from (b)

- G: original retrieved intersection grid.
- $\hbar^v = \{\mathbf{s_0}, \mathbf{s_1}, ..., \mathbf{s_{m\times m-1}}\}$: a matrix set where each element $\mathbf{s_j}$ is a 3×4 matrix with each row element forming a vertical De Bruijn subsequence $(a_{0j}, a_{1j}, a_{2j}, a_{3j})$ of length 4,
$$\mathbf{s_j} = \begin{bmatrix} a_{0j} & a_{1j} & a_{2j} & a_{3j} \\ a_{0j} & a_{1j} & a_{2j} & a_{3j} \\ a_{0j} & a_{1j} & a_{2j} & a_{3j} \end{bmatrix}.$$
- $\mathbf{H}$: the function for calculating per-element Hamming distance between two matrices.

The outputs are:

- r_s^v, r_w^v, r_e^v and r_n^v.
- G': amended intersection grid with virtual neighbors.

The two matrices π^v and π^h are of size 3×4 and 4×3 for horizontal and vertical separately.

IV. EXPERIMENTS AND RESULTS

Our implementation setup is composed of a Point Grey FFMV2 color camera with a resolution of 640×480 and a Optima 66HD DLP projector. When the projector projects a color-coded light pattern onto a 3D object, the camera located at 1m away from the object captures its image. Both camera and projector are calibrated with their intrinsic and extrinsic parameters known. To validate the accuracy of our proposed method, a plaster bust with a complex surface is used in our experiments. The object and the captured intersection network are given in Figs. 3a and 3b. The network contains 62×60 color stripes. Due to the highly deformed surface, especially at the hair, eyes, and nose areas, there are several holes as depicted in Fig. 3c. Most of them are due to occlusion distortion. Note that with the help of the watershed algorithm, the impact of sharp depth discontinuity between the background and object is minimal. Our reconstruction results are given in Figs. 4a and 4b. In the experiment, the total number of intersections in the projected De Bruijn grid (66×66) is 4356. However, four horizontal and six vertical stripes are out of the cameras FOV, resulting in the size of the captured grid 62×60 with 3720 intersections. Due to occlusion distortion, there are 393 intersections missing, especially in the areas of hair, eyes and boarder of the bust and background. With this ground truth, our method is able to recover 3327 intersections with 267 labeled wrong and 110 containing incomplete neighborhood information. This corresponds to 88.6% correct assignment. The reconstructed point cloud and mesh are shown in Fig. 4.

With the focus on local optimization, such as the vote majority rule for color detection and correction, and De Bruijn-based Hamming distance to improve neighborhood information of intersections, our method achieves the best speed in comparison to existing one-short methods [7], [9]. In unoptimized C++ code, the running time for 3327 grid intersections was about 1 second on a desktop with an Intel i7-2600k CPU with four cores (3.40 GHz) along with 8G DDR3 1600 memory, and the operating system is Windows 7 64bit version.

V. CONCLUSION

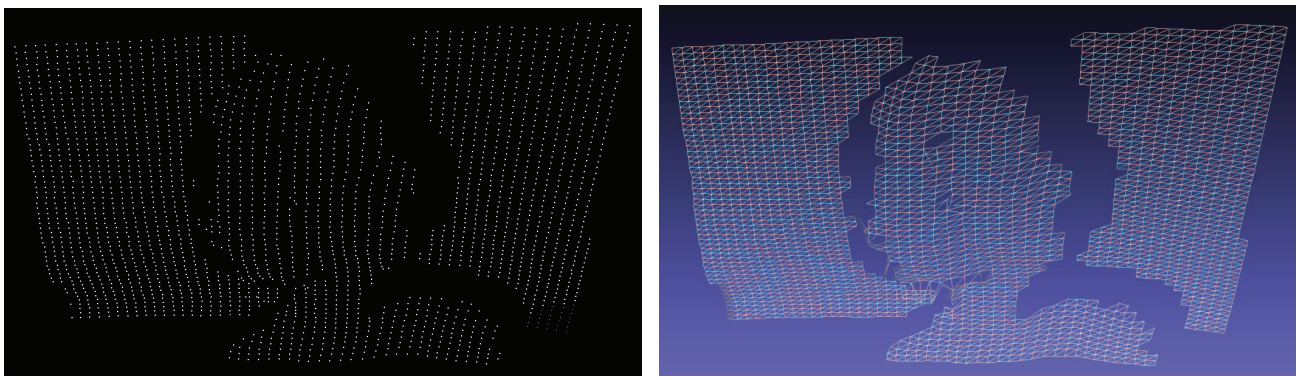
This paper presents a fast one-shot real-time 3D reconstruction method using De Bruijn encoded color pattern. Unlike previous one-short approaches, our formulation focuses on local optimization, where a Hamming distance minimization method is used for finding missing topological information and a special vote majority rule is applied for color detection and correction. The proposed method is approved to be efficient with a desirable accuracy achieved, even when applied to a very complicated scene. Note that with the focus on mobile applications, such as mobile augmented reality [2], we emphasize more on speed over accuracy as 3D information is only used for rendering a live model based on limited feature points in the application.

Our experimental results also present some limitation of our method, particularly when dealing with complex surfaces, such as the hair gaps on the bust in our experiments. How to tackle those situations, and utilize the proposed method for real-time object tracking are our future work.

ACKNOWLEDGMENT

This work is supported in part by the National Science Foundation grant #OCI-1041306.

Authorized licensed use limited to: Lakehead University. Downloaded on September 30,2026 at 23:49:39 UTC from IEEE Xplore. Restrictions apply.



(a) Point cloud (b) Mesh

Fig. 4: Reconstructed point cloud (a) and a mesh (b)

REFERENCES

[1] S. C. Park and M. Chang, "Reverse engineering with a structured light system," *Computers & Industrial Engineering*, vol. 57, no. 4, pp. 1377–1384, 2009.

[2] M. Torres, R. Jassel, and Y. Tang, "Augmented reality using spatially multiplexed structured light," in *Mechatronics and Machine Vision in Practice (M2VIP), 2012 19th International Conference*, Nov 2012, pp. 385–390.

[3] O. V. Olesen, R. R. Paulsen, L. Hojgaard, B. Roed, and R. Larsen, "Motion tracking for medical imaging: a nonvisible structured light tracking approach," *Medical Imaging, IEEE Transactions on*, vol. 31, no. 1, pp. 79–87, 2012.

[4] S. P. McPherron, T. Gernat, and J.-J. Hublin, "Structured light scanning for high-resolution documentation of¡ i¿ in situ¡/i¿ archaeological finds," *Journal of Archaeological Science*, vol. 36, no. 1, pp. 19–24, 2009.

[5] J. Salvi, S. Fernandez, T. Pribanic, and X. Llado, "A state of the art in structured light patterns for surface profilometry," *Pattern recognition*, vol. 43, no. 8, pp. 2666–2680, 2010.

[6] L. Zhang, B. Curless, and S. M. Seitz, "Rapid shape acquisition using color structured light and multi-pass dynamic programming," in *3D Data Processing Visualization and Transmission, 2002. Proceedings. First International Symposium on*. IEEE, 2002, pp. 24–36.

[7] A. Ulusoy, F. Calakli, and G. Taubin, "Robust one-shot 3d scanning using loopy belief propagation," in *Computer Vision and Pattern Recognition Workshops (CVPRW), 2010 IEEE Computer Society Conference on*, June 2010, pp. 15–22.

[8] H. Kawasaki, R. Furukawa, R. Sagawa, and Y. Yagi, "Dynamic scene shape reconstruction using a single structured light pattern," in *Computer Vision and Pattern Recognition, 2008. CVPR 2008. IEEE Conference on*, June 2008, pp. 1–8.

[9] S. Chen, Y. Li, and J. Zhang, "Vision processing for realtime 3-d data acquisition based on coded structured light," *Image Processing, IEEE Transactions on*, vol. 17, no. 2, pp. 167–176, Feb 2008.

[10] P. Fechteler and P. Eisert, "Adaptive color classification for structured light systems," in *Computer Vision and Pattern Recognition Workshops, 2008. CVPRW '08. IEEE Computer Society Conference on*, June 2008, pp. 1–7.

[11] T. van Aardenne-Ehrenfest and N. de Bruijn, "Circuits and trees in oriented linear graphs," in *Classic papers in combinatorics*. Springer, 1987, pp. 149–163.

[12] J. Pagès, J. Salvi, and C. Matabosch, "Robust segmentation and decoding of a˜ grid pattern for structured light," in *Pattern Recognition and Image Analysis*. Springer, 2003, pp. 689–696.

[13] Y. Xu and D. G. Aliaga, "Modeling repetitive motions using structured light," *Visualization and Computer Graphics, IEEE Transactions on*, vol. 16, no. 4, pp. 676–689, 2010.

[14] S. Chen, Y. Li, and J. Zhang, "Realtime structured light vision with the principle of unique color codes," in *Robotics and Automation, 2007 IEEE International Conference on*. IEEE, 2007, pp. 429–434.

[15] C. Hilditch, "Comparison of thinning algorithms on a parallel processor," *Image and Vision Computing*, vol. 1, no. 3, pp. 115–132, 1983.

[16] L. Vincent and P. Soille, "Watersheds in digital spaces: an efficient algorithm based on immersion simulations," *IEEE transactions on pattern analysis and machine intelligence*, vol. 13, no. 6, pp. 583–598, 1991.

Authorized licensed use limited to: Lakehead University. Downloaded on September 30,2026 at 23:49:39 UTC from IEEE Xplore. Restrictions apply.